

02 · Does the offer work better for some segments? — segment effects (pathmc)

July 12, 2026

The business decision. The email lifts spend *on average* (that’s the ATE — Average Treatment Effect — from notebook 01). But an average can hide everything that matters: maybe the email really moves **high-value, highly-engaged** customers and barely touches the rest. If the segments genuinely differ, we should write segment-specific send rules; if the apparent “gap” is just noise, chasing it would be overfitting our marketing to randomness. Either way we want to know **what it costs to treat everyone as the average**.

0.0.1 The concept: moderation and the CATE

When a treatment’s effect *depends on who receives it*, we say the effect is **moderated** by the customer’s characteristics, and the characteristic (here, being high-value, or an engagement score) is a **moderator**. Formally we’re estimating the **CATE** — the *Conditional Average Treatment Effect*, $\tau(x) = \mathbb{E}[Y(1) - Y(0) \mid X = x]$: the average email effect *among customers who look like x* . ($Y(1), Y(0)$ are the two potential outcomes — spend if emailed / not — from notebook 01; $\mathbb{E}[\cdot]$ is “average over customers.”) The ATE is just the CATE averaged over everybody.

0.0.2 The tool: a structural model + the do-operator

Notebook 01 used flexible trees (BART) to get the CATE. Here we use a different, more *interpretable* approach — a **structural causal model** via the **pathmc** library. We write down the equation we believe generated spend (including an **interaction term** — a feature that multiplies the treatment, which is exactly how “the effect depends on a moderator” is encoded), fit it, and then ask the model counterfactual questions with the **do-operator**: **do(email=1)** means “*intervene* to email this customer,” as opposed to merely *observing* emailed customers (who differ in other ways). The difference in predicted spend between **do(email=1)** and **do(email=0)**, at a given x , *is* the CATE at x .

The single most important output is the **posterior of the interaction coefficient** (the **posterior** is the distribution of plausible values for a quantity after the data have spoken — wide when we’re unsure, tight when we’re sure; every “is it real?” question below is read off it): if it’s clearly away from zero, the segments *really* respond differently; if it straddles zero, we can’t tell the gap from noise — so we shouldn’t build segment rules on it (and might instead run a bigger test). That’s the “is the difference real?” test, answered with uncertainty.

0.0.3 How this notebook is organised

The repo’s fixed **7-step contract** — 1 question · 2 simulate a known truth · 3 identify · 4 estimate · 5 validate · 6 decide in € · 7 caveats — plus the depth a skeptic will ask for at each

step:

- **Step 3** draws the DAG and walks the three identification assumptions one by one;
- **Step 4** puts the likelihood *and* the priors on the table, checks the model can regenerate the data (a posterior predictive check), and climbs the classic **pooling ladder** — full pooling vs no pooling vs the interaction model (§4b);
- **Step 5** grades recovery against the planted truth (one sample, then many), then covers the two ways segment analysis goes wrong in practice — **subgroup fishing** and **underpowered interactions** (§5c) — and stress-tests a **wrongly specified** model plus a model-free cross-check (§5d);
- **Step 6** turns posteriors into send rules and prices what pooling costs, *with uncertainty on that headline number*, closing with a signable decision memo (§6b).

A note on intervals: coefficient tables print ArviZ’s default **94% HDI** (highest-density interval; defined in §4); plots and CATE read-outs use **90%** bands (the cookbook’s decision convention). Each printout names its level, so the mix is cosmetic, never ambiguous.

0.1 2 · Simulate a ground truth

Customers are **high- or low-value** (`prior_value`) and also carry a continuous **engagement** score. We plant a real, layered moderation: the email lifts low-value customers by **€3** and high-value by **€12 at zero engagement**, **plus** a smooth **+€8 x engagement** slope on top — so at the **average** engagement (0.5) the segment effects work out to **€7** (low) and **€16** (high), which is what the recovery below is graded against. The true per-customer effect is $\tau = 3 + 9 \cdot \text{is_high} + 8 \cdot \text{engagement}$ — a genuine, sizeable, and *continuous* heterogeneity, spanning €3 (low value, zero engagement) to ~€20 (high value, full engagement). `prior_value` also drives spend directly (must be included); there’s a background **trend**.

On real data. There’s no single public dataset for this exact setup, but the swap is trivial: your **own CRM / campaign log** already has a segment column (`prior_value`), covariates, a treatment flag (was the customer emailed) and an outcome (spend). Point the model at those columns instead of the simulator. The one requirement is that the email was assigned randomly *or* that you’ve measured the confounders (notebook 05) — otherwise the “segment effects” are contaminated by who-got-emailed.

The data-generating model — exactly what `dgp.segment_customers` implements (defaults & seed in `src/cmp/dgp.py`). $H = \mathbf{1}[\text{prior_value} = \text{high}]$ with $P(H=1) = 0.4$, engagement $E \sim \text{Beta}(2, 2)$, background trend $G \sim \mathcal{N}(50, 10^2)$, and a **randomized** email $T \sim \text{Bernoulli}(\frac{1}{2})$:

$$\begin{aligned} \tau(H, E) &= 3 + 9H + 8E && \text{(the planted CATE)} \\ \text{spend} &= 5 + \tau(H, E)T + 18H + 4E + 0.3G + \varepsilon, && \varepsilon \sim \mathcal{N}(0, 6^2). \end{aligned}$$

Note the two distinct roles of H : the $18H$ term is prior value’s **direct** effect on spend (high-value customers spend more regardless — why the model must include it), while the $9H$ inside τ is its **moderation** of the email effect — the thing this notebook estimates. At the mean engagement $E = 0.5$ the segment truths quoted above are $3 + 8(0.5) = 7$ € (low) and $3 + 9 + 8(0.5) = 16$ € (high).

TRUE segment-average effects (at mean engagement): {'low': 7.0, 'high': 16.0}
 TRUE effect spans €3.0 ... €19.9 across customers

	email	prior_value	engagement	trend	spend	is_high
0	0.0	low	0.918802	58.466377	23.337758	0.0
1	0.0	low	0.651170	60.223782	19.057907	0.0
2	1.0	high	0.242720	53.530129	50.963256	1.0
3	0.0	low	0.119019	37.977148	12.868787	0.0
4	1.0	low	0.488445	61.807028	29.494100	0.0

0.2 3 · Identify — the CATE as a functional of the model

Before fitting anything, we pin down *exactly* what we’re estimating (the CATE is a *functional* — a summary quantity computed from the fitted model) and whether it’s recoverable.

The estimand. The conditional effect, written with the do-operator (intervene, don’t just observe):

$$\tau(x) = \mathbb{E}[\text{spend} \mid \text{do}(\text{email}=1), x] - \mathbb{E}[\text{spend} \mid \text{do}(\text{email}=0), x].$$

Where heterogeneity lives. We assume spend is generated by the structural equation (with $e=\text{email}$, $v=\text{is_high}$, $g=\text{engagement}$)

$$\text{spend} = \beta_0 + \beta_1 e + \beta_2 v + \beta_3 (e \cdot v) + \beta_4 (e \cdot g) + \dots$$

Take the difference between $e=1$ and $e=0$ and everything not multiplied by e cancels, leaving

$$\tau(x) = \beta_1 + \beta_3 v + \beta_4 g.$$

So the email effect is a **baseline** β_1 **plus moderation terms: all the heterogeneity is carried by the interaction coefficients** β_3, β_4 . If those were zero, τ would be the same for everyone and `cate()` would collapse to `ate()`. This is why “is the effect heterogeneous?” is literally the question “is β_3 (or β_4) non-zero?” — which we answer with its posterior in Step 5.

Identification — the three assumptions, one by one. Writing the estimand with $\text{do}(\cdot)$ doesn’t make it recoverable; three assumptions do. Each gets a name, a formula, and its marketing meaning:

1. **Unconfoundedness** (“no unmeasured common causes”): $\{Y(0), Y(1)\} \perp T \mid X$ ($\perp$ means “is independent of”) — *who got the email carries no information about how they would have spent anyway*. Here it holds **by design**, because email was randomized. One segment-specific subtlety worth a sentence: in observational data, a hidden driver correlated with the **moderator** (say, app usage driving both engagement and email opens) doesn’t just bias the average effect — it bends the *interaction* itself, manufacturing fake heterogeneity. Randomization protects the interactions too.
2. **Positivity within every segment**: $0 < P(T=1 \mid X=x) < 1$ for every x we quote a CATE for. Randomization gives exactly $\frac{1}{2}$ everywhere; on real CRM data thin segments are routinely *all-treated* (“of course every high-value customer got the campaign”), and a segment CATE there is pure extrapolation. It’s a one-line check — we run it below because on real data you always should.

3. **SUTVA** (no interference, one treatment version): one customer's email doesn't move another customer's spend. The marketing failure mode is *forwarding / household spillover* — and note it typically happens **within** a segment (family members share a value tier), contaminating exactly the segment averages this notebook estimates.

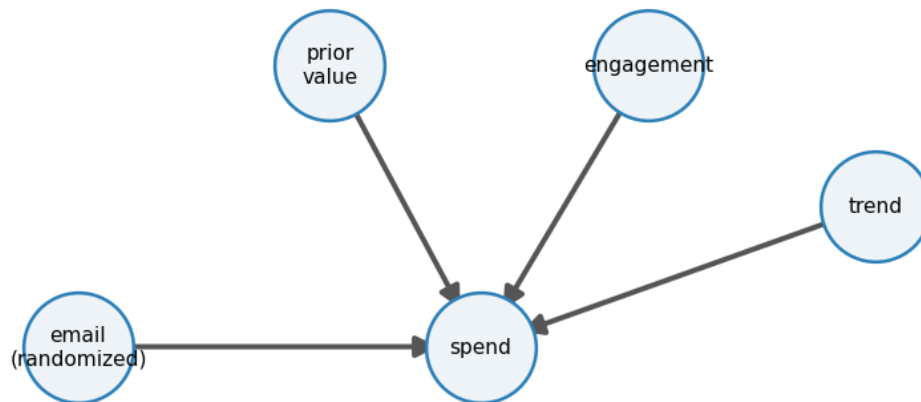
One extra requirement, specific to CATEs: condition on *pre-treatment* covariates only. `is_high` and `engagement` must be measured *before* the send. Slicing by a post-treatment variable (“the effect among those who opened”) conditions on a consequence of the treatment and wrecks the comparison — that's notebook 04's collider material. Relatedly, moderation is *descriptive of who responds*, not a claim that changing the moderator would change responsiveness (Step 7 returns to this).

The DAG below draws the assumption set: four arrows into `spend` and — the visual signature of randomization — **no arrow into email**. One honest limitation to notice now: a DAG encodes *which* arrows exist, not their functional form. The moderation β_3, β_4 lives *inside* the email->spend arrow, invisible to the graph — remember that in §5d, when the graph-based falsification test fails to catch a wrong functional form.

P(emailled) by segment: high: 0.48 low: 0.51
P(emailled) by engagement tercile: low: 0.46 mid: 0.52 high: 0.52
-> every subgroup has both arms in force (all shares near 0.5): positivity holds, so each segment's

CATE is a comparison, not an extrapolation. On real data, a share near 0 or 1 voids that segment.

The declared DAG: four causes of spend, no arrows INTO email



nothing points at email = randomization; the software's empty adjustment set (next cell) is this picture, read off algorithmically

admissible adjustment set(s) for email->spend: [set()]
identifiable: True

(An empty adjustment set [set()] means “condition on nothing” — correct, because randomization already made email independent of everything. In an *observational* version we'd see the

confounders listed here, exactly as in notebook 05.)

0.3 4 · Estimate — fit the structural model

What `m.fit()` actually samples. Writing the fitted model out once keeps the Bayesian machinery honest (e =email, v =is_high, g =engagement, t =trend; i indexes customers):

$$\text{spend}_i \sim \mathcal{N}(\mu_i, \sigma^2), \quad \mu_i = \beta_0 + \beta_1 e_i + \beta_2 v_i + \beta_3 e_i v_i + \beta_4 e_i g_i + \beta_5 g_i + \beta_6 t_i,$$

$$\beta_k \sim \mathcal{N}(0, 10^2) \text{ for every coefficient (intercept included),}$$

$$\sigma \sim \text{HalfNormal}(1) \quad (\text{pathmc's defaults — printed, not trusted, below}).$$

The likelihood says spend is Gaussian noise around a mean that is linear in the parameters; the priors are the package defaults, which we print from the model object right after sampling rather than quoting from documentation. Are they *sensible*? A prior is only “weak” or “strong” **relative to the likelihood’s precision**, never to the raw data scale: with $n = 1,500$ rows the data speak so loudly that even the alarming-looking HalfNormal(1) on a noise level that is truly ~€6 gets overruled — the posterior for `sigma_spend` lands where the data put it (printed below). The cell after the fit makes the same point in reverse with a deliberately dogmatic prior that *rivals* the likelihood’s precision and therefore wins. When segments are thin, the per-segment likelihood is weak and priors regain their power — that is exactly nb03’s partial pooling, previewed in §4b.

The `effects_summary()` table below is the direct read-out: each row is a β with its posterior mean and 94% HDI (*Highest Density Interval* — the tightest interval holding 94% of the posterior mass). The two rows to watch are `b_ev` and `b_eg` — the interaction terms. If their intervals sit clearly away from zero, the segments genuinely respond differently; that’s the statistical backing for writing segment-specific rules rather than one blanket policy.

segment model convergence: max r-hat 1.010 - min ESS 403 - divergences 0

	mean	sd	hdi_3%	hdi_97%
name				
b_e	2.68	0.80	1.20	4.03
b_v	18.41	0.43	17.62	19.24
b_ev	8.08	0.61	6.96	9.24
b_eg	10.05	1.37	7.56	12.52
b_g	3.95	0.94	2.27	5.77
b_tr	0.29	0.02	0.27	0.32

Reading the sampler’s health check. Those three numbers say whether the MCMC sampler — the algorithm that draws samples from the posterior — actually converged. **R-hat** compares the variance *within* each chain to the variance *across* chains; 1.00 is perfect and ≤ 1.01 is **the usual pass bar** (above it the chains disagree and the numbers aren’t safe to read). **ESS** (effective sample size) is how many *independent* draws our autocorrelated chains are worth — a few hundred is ample for a posterior mean or a 94% interval. **Divergences** are steps where the sampler broke down and silently distrusts that region; **you want 0**. Here all three pass, so the posterior below is trustworthy. (Under the FAST teaching profile the chains are deliberately short, so R-hat can drift to ~1.02 and PyMC may print a “problems during sampling” notice — a benign artifact of

the small draw count that clears in a FULL run; §5b's multi-seed recovery is the real backstop that the estimates are sound.)

pathmc's default priors for this model:

Priors:

```
beta_spend: Normal(mu=0, sigma=10)
sigma_spend: HalfNormal(sigma=1)
```

posterior sigma_spend ~ €5.94 (true noise €6): the HalfNormal(1) prior 'wanted' a far smaller

sigma, but 1,500 rows of likelihood overruled it - weak/strong is about the likelihood, not the prior.

same data, beta prior Normal(0, 0.1): high-value CATE collapses to €0.69 (default priors: €15.79)

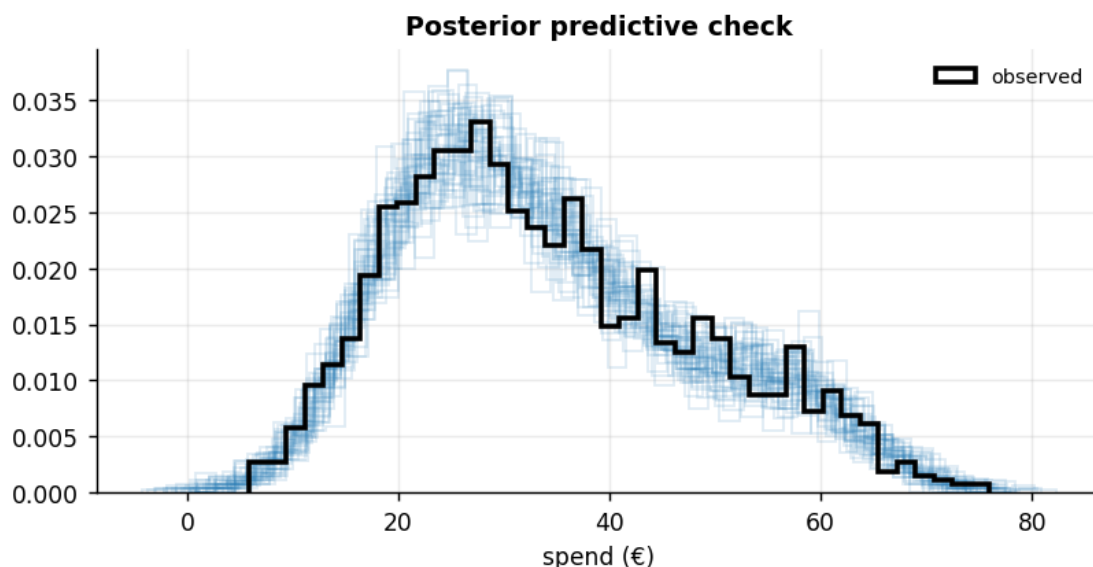
-> the posterior is always prior x likelihood; the defaults were harmless only because n made the

likelihood sharp. With dozens of THIN segments the per-segment likelihood is weak - exactly when

priors must be chosen (partial pooling, nb03), not defaulted.

Model criticism — the posterior predictive check (PPC). Before we read effects off a structural model, we ask whether it can even *regenerate the data it was fit to*. We draw replicated spend vectors from the fitted model — $y_i^{rep} = \mu_i^{(s)} + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \sigma^{(s)})$, one replicate per posterior draw s — and overlay them on the observed spend distribution, the same discipline as notebook 01. If the real data looked nothing like what the model can generate (wrong spread, missed skew), the effect estimates downstream would be untrustworthy no matter how tight their intervals. A model that cannot reproduce the outcome's shape has no business issuing counterfactuals about it.

PPC: 90% of observed spend falls inside its replicates' 5-95% band (~90% if calibrated) - no gross misfit; the model can regenerate the data it explains.



0.3.1 4b · Three ways to get a segment effect — and why the interaction model wins

The interaction model is not the only way to answer “does the effect differ by segment?” — it is the best of three classic options, and seeing all three on one axis is the fastest way to understand what the interaction terms buy. Writing τ_s for segment s ’s effect and n_s for its row count:

- (a) full pooling: $\tau_s \equiv \tau \quad \forall s$
- (b) no pooling: τ_s estimated from the n_s rows of segment s alone
- (c) interaction: $\tau_s = \beta_1 + \beta_3 s$, nuisances shared.

- **(a) Full pooling** — the same structural model with *no* interaction terms. Everyone is assigned the ATE. This is “treat everyone as the average” *as a model* rather than as a policy — and if real heterogeneity exists it is wrong for **every** segment at once.
- **(b) No pooling** — split the sample and fit `spend ~ email + engagement + trend` separately inside each segment. Unbiased for each segment’s mean effect, but wasteful: each half re-estimates σ , the engagement slope and the trend slope from its own rows only — and there is no single posterior for the *gap* between segments, nor for a continuous moderation profile that crosses them.
- **(c) The interaction model** already fitted. Because the binary moderator is *saturated* by the $e \cdot v$ term — that term gives it one free parameter per level, so it can hit each segment’s mean exactly, just as separate fits would — it reproduces the no-pooling segment *means* — while sharing every nuisance parameter across segments and carrying the gap (β_3) and the engagement slope (β_4) as first-class posterior quantities.

With two large segments the pure *efficiency* difference is modest — both halves still have hundreds of rows, and the printed interval widths below quantify exactly how much (or little) sharing nuisances buys here. The trade turns brutal as segments multiply and thin: with 20 segments of 75 customers, every no-pooling fit is noise. The cure then is **partial pooling** — treat the segment effects as draws from a population,

$$\tau_s \sim \mathcal{N}(\mu_\tau, \sigma_\tau^2),$$

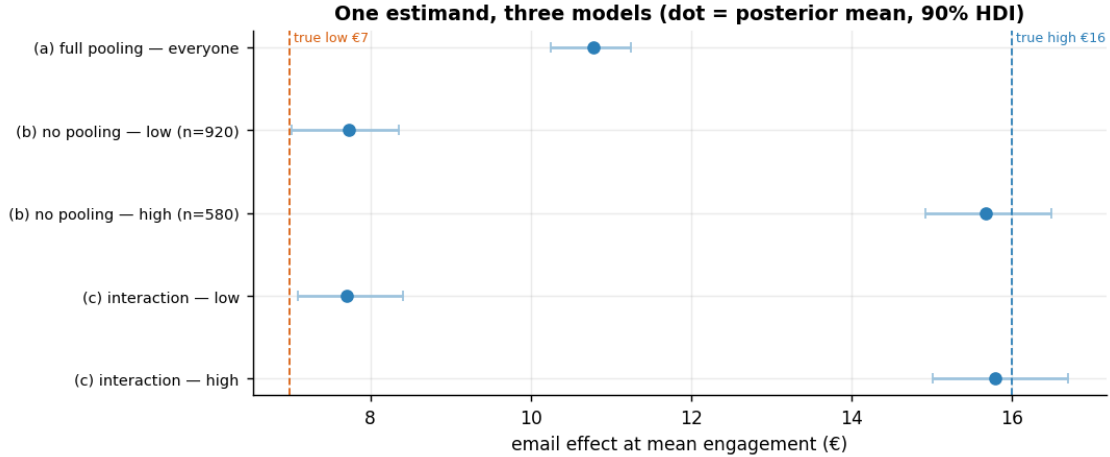
where μ_τ is the average effect across segments and σ_τ is how much the segments genuinely differ — estimating σ_τ from the data is what decides how hard to shrink — so noisy, thin segments are shrunk toward the population mean and borrow strength from fat ones. That is notebook 03’s random-slopes machinery; here we’ve just turned the usual throwaway caveat into an equation.

full pooling answers €10.78 for BOTH segments (truths €7 / €16) - wrong for each
90% interval widths - low : no-pooling €1.34 vs interaction €1.30

high: no-pooling €1.57 vs interaction €1.68

similar widths here (both segments still have hundreds of rows); the no-pooling penalty grows as

segments thin - and only the interaction model carries a posterior for the GAP (b_ev) and the engagement slope (b_eg) at all.



Reading the forest plot. Full pooling (a) lands between the two truths and is therefore wrong for *both* segments at once — too optimistic about low-value customers, too pessimistic about high-value ones. This is the statistical face of the “cost of pooling” that Step 6 prices in campaign euros: the bias doesn’t average out, it mis-targets both halves of the base. No pooling (b) is centred correctly but pays for re-learning σ and the nuisance slopes inside each half — with segments this large the price is small (compare the printed widths), which is itself worth knowing: the interaction model’s advantage here is **coherence**, not raw precision. Only (c) delivers, from one fit, the two segment effects *plus* a posterior for their difference (β_3 — Step 5’s “is the gap real?”) *plus* the continuous engagement profile (β_4) — quantities (b) can only bolt together from two independent posteriors and (a) denies exist. One estimand, three bias–variance trade-offs; (c) sits at the sweet spot until segments get thin, at which point partial pooling (nb03) takes over.

DAG falsification — the check you’d run on real CRM data. Recovery-vs-truth is only available in simulation; on real data you instead test the DAG’s *implied* conditional independences against the data — a wrong DAG shows up as an independence the graph predicts but the data violate.

DAG falsification: 6 implied independences tested, 0 violated at alpha=0.05 (graph consistent with the data).

0.4 5 · Validate — recover the effects, the gap, and the moderation profile

Four checks:

1. **Segment CATEs** vs the planted **€7 / €16** (the low/high segment averages at mean engagement, 0.5) — with **credible intervals** (a credible interval is the Bayesian analog of a confidence interval: a range the parameter falls in with the stated probability, given the data).
2. **Is the gap real?** the interaction posterior β_3 ; if it clearly excludes zero, the segments truly differ (not noise).
3. **The continuous moderation profile** — CATE as a smooth function of engagement, recovered vs the true $3 + 9 \cdot \text{is_high} + 8 \cdot \text{engagement}$ line. This is the payoff of modelling the continuous moderator rather than crudely binning customers into two groups.

4. **Per-customer recovery** — the effect predicted for each individual customer (from the fitted moderation coefficients) against their true effect, on the 45° line. Recovering *segment averages* is easy; recovering the effect for each *individual* is the real test.

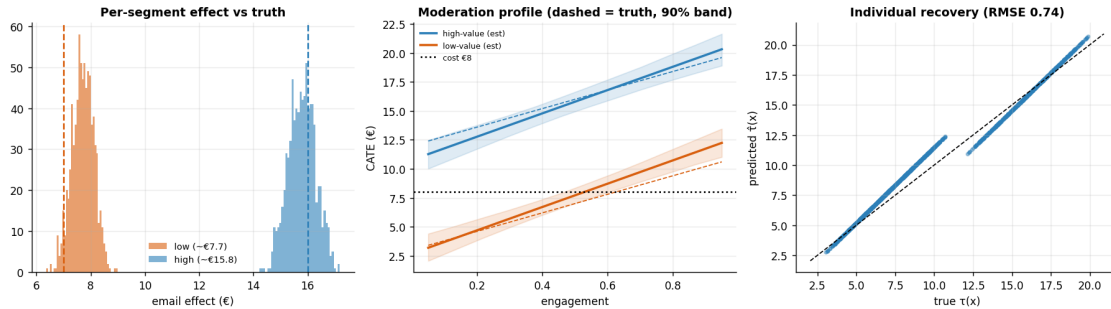
ATE (pooled) €10.80 94% HDI [€10.24, €11.39]

CATE low (eng .5) € 7.71 (true 7.0) 94% HDI [€6.96, €8.45]

CATE high (eng .5) €15.79 (true 16.0) 94% HDI [€14.90, €16.78]

Interaction beta(email:is_high) mean €8.08 (true 9.0), $P(>0) = > 0.999 \rightarrow$ gap is REAL

Interaction beta(email:engagement) mean €10.05 (true 8.0), 90% credible interval [€7.75, €12.14] — covers the planted €8.0 on this draw; the moderation panel below shows where that slope error pulls the fitted line off the truth



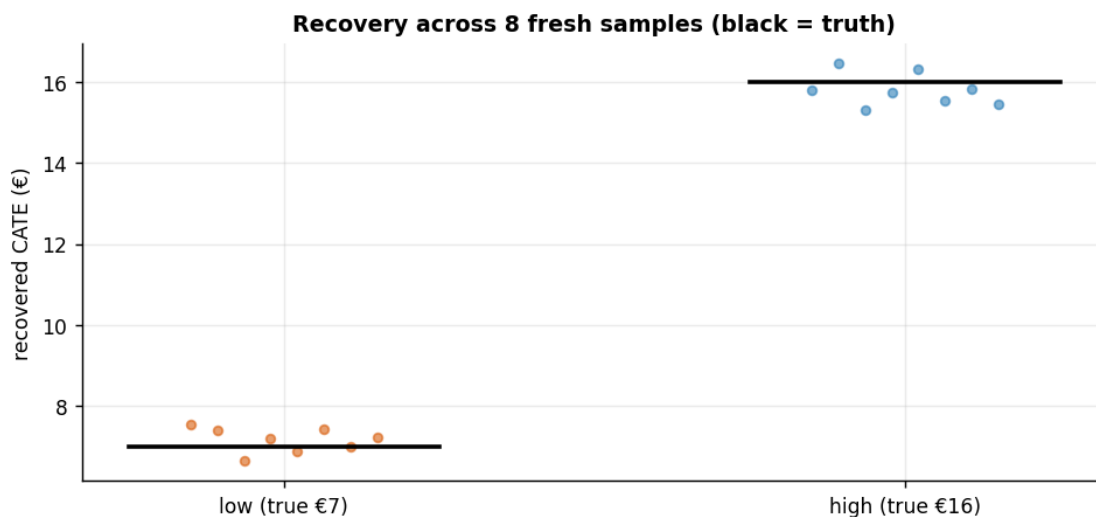
How to read those three panels. *Left* — the two segment-effect posteriors land close to their planted truths (dashed lines) and barely overlap, so the model both **recovers** the effects and **separates** the segments (the low-value posterior runs ~€0.7 high on this draw — §5b shows the bias vanishes across fresh samples). *Middle* — the estimated CATE (solid) tracks the true $3 + 9 \cdot \text{is_high} + 8 \cdot \text{engagement}$ line (dashed) over most of the engagement range — though this draw's interaction slope runs high (~€10 fitted vs €8.0 planted — the exact figure is printed above), so the fitted line pulls above the truth at high engagement: modelling the *continuous* moderator paid off, because the effect clearly rises with engagement rather than being a flat per-segment number. Note where each line crosses the €8 cost line — that's the **break-even engagement**, the level above which a customer becomes worth emailing; we quantify it (with a 94% HDI) in the decide step below. *Right* — predicted vs true effect *per customer* hugs the 45° line (low RMSE), confirming recovery at the individual level, not just on segment averages.

0.4.1 5b · Recovery across many seeds — unbiased *and* calibrated?

A single sample can land a little off; the real test of an estimator is whether it is **centred on the truth over repeated samples** (unbiased, not just lucky here) and whether its intervals **cover** the truth at their stated rate. We refit on many fresh samples and check both. (The per-seed refits are deliberately small, fast fits — enough for the bias/coverage tallies we read off them; their sampler chatter is silenced below.) We track three quantities per refit: the two segment CATEs *and* the low-value **break-even engagement** g^* — a *ratio* of coefficients, the estimand the decision step will warn is fragile near a crossing — so that when Step 6 reads its break-even off one sample, we already know how that ratio behaves across many.

low -value: mean €7.16 (true €7) bias +0.16 sd 0.29 · 90% HDI covers truth in 8/8 seeds
 high-value: mean €15.81 (true €16) bias -0.19 sd 0.38 · 90% HDI covers truth in 8/8 seeds
 g* break-even : mean 0.609 (true 0.625) bias -0.016 sd 0.037 · 94% HDI covers truth in 8/8 seeds

-> the ratio's coverage sits near nominal across fresh samples: a one-sample miss (see the decide step) reads as bad luck near the crossing, not systematic under-coverage.



0.4.2 5c · Two ways segment analysis goes wrong: fishing, and underpowered interactions

The single most famous failure mode of segment analysis isn't a bias in any estimator — it's **subgroup fishing** (multiple comparisons). Slice a campaign twenty ways and *something* will look responsive by chance; the analyst then writes a confident slide about “segment 13”. Before trusting our own gap, we stage the trap on this very dataset: keep the real outcome and the real randomized email, but generate **20 pure-noise binary “segments”** — coin-flip labels with, by construction, zero true moderation — and test each one's effect gap the way a naive analyst would (a 95% interval on the treated-minus-control difference between the two halves of each label). Each individual test is honest; **the batch is not**: 20 independent nulls at the 5% level yield about one false “discovery” on average.

The second failure mode is the mirror image: real interactions are **intrinsically hard to detect**. In a balanced 50/50 test split into two equal segments, the segment *gap* is the difference of two segment effects, so its sampling variance is the **sum** of theirs — and each segment effect is estimated from half the sample (the variance of an estimate scales like $1/n$, so halving the sample doubles it):

$$\text{Var}(\hat{\beta}_3) = \text{Var}(\hat{\tau}_{\text{high}}) + \text{Var}(\hat{\tau}_{\text{low}}) \approx 2 \text{Var}(\hat{\tau}_{\text{pooled}}) + 2 \text{Var}(\hat{\tau}_{\text{pooled}}) = 4 \text{Var}(\hat{\tau}_{\text{pooled}}),$$

where $\hat{\beta}_3$ is the interaction (gap) estimate and $\hat{\tau}_{\text{pooled}}$ the ATE estimate on the same n . The gap's standard error is therefore $\sim 2\times$ the ATE's. Required sample grows with $(\text{SE}/\text{effect})^2$, so a gap that carries $2\times$ the SE *and* — being half the size of the main effect — half the signal needs $(2/\frac{1}{2})^2 = 16\times$ the rows for equal power. That is Gelman's rule of thumb: you need $\sim 16\times$ the sample to detect an interaction *half* the size of the main effect with the same power. Interactions whisper; averages shout. We verify the 2x on our own posterior below, then use the same arithmetic to size the follow-up test that Step 6's cautious low-value verdict calls for.

1 of 20 pure-noise moderators produce a 95%-significant effect gap (~ 1 expected by chance) - 'responsive segments' manufactured from coin flips.

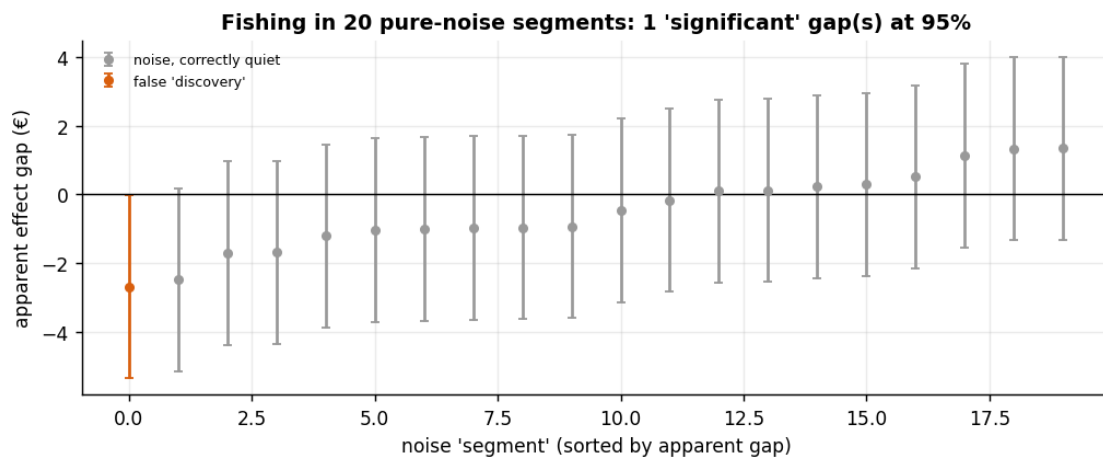
posterior sd - ATE: €0.32 segment gap b_ev: €0.61 ratio 1.9x (theory for a clean 50/50 split: 2x)

low-value CATE €7.71 vs cost €8: posterior sd €0.40 on $n = 1,500$.

posterior sd shrinks like $\sqrt{n_0/n}$; IF the truth sits at today's mean, $n \sim 7,758$ customers

puts 95% of the low-value posterior on one side of the cost line - 'run a bigger test' now has a

number attached (the closer the effect is to the cost, the more brutally that number grows).



Read-out. The batch of noise segments delivers what multiplicity theory promises — on the order of one sham “responsive segment” per twenty tests (any orange interval excludes zero purely by luck; the count is printed in the title). The defences, in order of cheapness: **pre-register** the segments you will act on before the campaign runs; report **posterior probabilities** rather than one-shot significance stars; and when you genuinely must scan many segments, use **shrinkage** / **partial pooling** (nb03), which pulls each segment's estimate toward the population mean in proportion to its noisiness — fishing dies when noisy estimates aren't allowed to stand alone. Note what this notebook's own analysis did: *two pre-declared moderators* (value tier, engagement), each with a prior business rationale — that design choice, more than any model, is the protection.

The power arithmetic explains Step 6's caution about the low-value segment. The quantity that decides whether segments *differ* (β_3) is about twice as noisy as the quantity that decides whether

the campaign *works* (the ATE) — on the same data (the printed ratio). Detecting that a gap exists is easy here only because the planted gap is huge; resolving a **borderline** segment call — low-value’s effect against the €8 cost — needs the multiplied sample the sizing line prints. Budget accordingly: an A/B test sized to measure the average is usually ~4x too small to measure *who* it works on.

0.4.3 5d · What if the model is wrong? Misspecification stress + a model-free cross-check

Time for an uncomfortable admission: the structural model recovered the truth partly because **we handed it the true functional form** — the simulator’s linear `email:engagement` moderation was copied straight into `spec`. On real data nobody hands you the form; you guess it. So we do what an honest analyst must: **break the model on purpose and watch what fails**. We refit with the engagement moderation *omitted* (`spec_bad` drops `email:engagement` — the guess “engagement doesn’t moderate the email”, which would sound perfectly reasonable before seeing the truth) and ask three questions: what happens to the *segment-level* answers? what happens to the *individual-level* answers and the break-even rule? and — the sharpest question — **does the DAG-falsification test from Step 4 catch the error?**

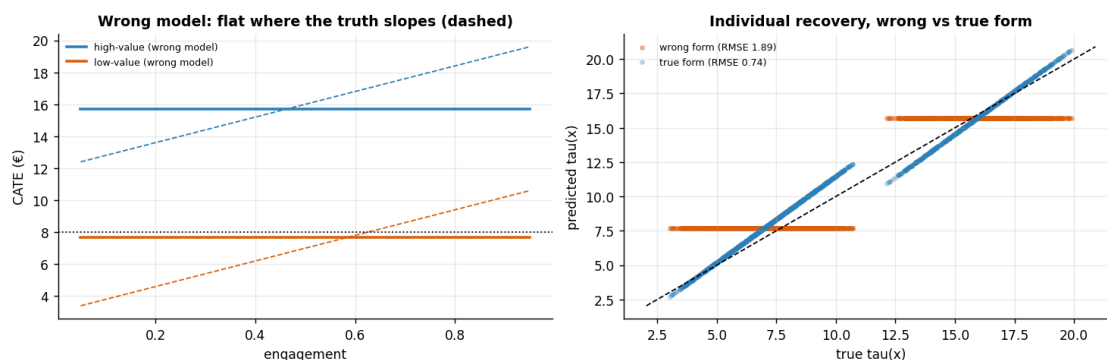
segment CATEs at mean engagement - wrong model: low €7.67 / high €15.72 (true-form model: €7.71 / €15.79)

per-customer recovery RMSE: €0.74 (true form) -> €1.89 (moderation omitted)

break-even engagement $(\text{COST} - b_e)/b_{eg}$: UNANSWERABLE - the wrong model has no b_{eg} , so Step 6's

'send low-value above engagement g^* ' rule cannot even be written down.

DAG falsification on the wrong model: 6 implied independences tested, 0 violated
-> the graph test does NOT flag it - expected, and the lecture point below.



What broke, and what didn’t. The wrongly-specified model still gets the *segment-level* answers nearly right (first printed line): omitting the engagement moderation folds its average contribution (~ €8 x mean engagement) into the baseline email coefficient, so at the segment level the error largely cancels. What breaks is everything *individual*: the per-customer recovery error inflates sharply (printed RMSEs), the moderation profile is flat where the truth slopes — the model now over-rates disengaged customers and under-rates engaged ones in *both* tiers — and the break-even engagement cutoff, Step 6’s operational deliverable, is not even expressible. **And the DAG test**

stayed silent. `test_implications` checks the *conditional independences implied by the graph*, and the wrong model has the *same graph* — the same nodes and arrows — as the right one; only the functional form on the email->spend arrow changed, which no independence test can see. Graph checks falsify your **arrows**; they cannot falsify your **algebra**. The tools for the algebra are the PPC (Step 4), multi-seed recovery when a simulator exists (§5b), and — on real data — a flexible, **model-free cross-check**, which we run next: a BART T-learner (notebook 01’s machinery) that learns the two spend surfaces with no functional-form commitment at all. If its segment effects land near the structural model’s, the structure isn’t doing the work — the data are.

low -value CATE near engagement 0.5 - BART T-learner € 7.69 vs structural € 7.71

high-value CATE near engagement 0.5 - BART T-learner €15.51 vs structural €15.79

-> a model-free learner lands on the same segment effects (within BART's wobble): the structure isn't doing the work, the data are.

0.5 6 · Decide, in euros — segment rules, and what pooling costs

Now the payoff. First we turn the per-segment effects into concrete **send rules** using the honest probability criterion from notebook 01 — target a segment only where **P(effect > cost)** is **high** (not merely where the mean beats cost). Then we answer the question that justifies this whole analysis: **how much money does segmenting actually make versus treating everyone as the pooled average?**

We compare four policies on the *known* per-customer truth (only possible in simulation), forming a ladder of increasing sophistication: **pooled** (one all-or-nothing decision on the ATE) -> **segment rules** (one decision per segment) -> **individual** (a per-customer decision on $\hat{\tau}(x)$) -> **oracle** (the unbeatable rule that knows every true effect). The gaps between the rungs are, literally, the euros that finer targeting is worth.

Two rules, two questions. The four-policy ladder below deliberately uses the risk-neutral posterior-**mean** rule (send whenever *expected* effect beats cost — profit-maximising for expected profit), while the per-segment SEND/TEST/SKIP table (printed first in the next cell’s output) uses the conservative **P(effect > cost) > 0.8** rule; the two answer different questions — how much money finer targeting is *worth* vs. which segments we are *confident enough* to act on now.

(When the table’s verdict for a segment is a hesitant one — a $P(\text{effect} > \text{cost})$ in the murky middle — §5c’s power arithmetic is what turns “run a bigger test” into an actual sample size.)

One discipline carried over from the validate step: the headline this section produces (“targeting is worth €X”) is itself an *estimate*. After computing it on the posterior mean, we recompute it **per posterior draw** — each draw implies its own send rules and its own realised profit — so the euro value of segmentation gets a credible interval and a $P(> 0)$ of its own, the same honesty we demand of every coefficient.

segment	effect	net_of_cost	P(effect>cost)	action
low-value	7.71	-0.29	0.24	SKIP
high-value	15.79	7.79	1.00	SEND

Realised profit by policy:

pooled (by ATE)	€3,682
segment rules	€4,602
individual tau_hat	€4,871
oracle	€4,923

Moving from pooled to individual targeting is worth €1,189 (32% uplift).

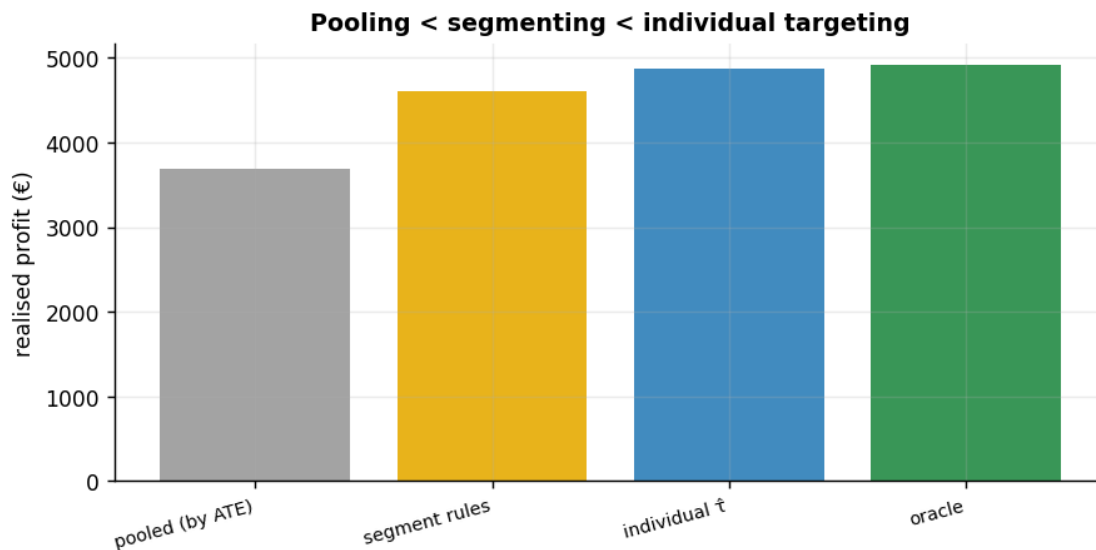
Value of targeting vs pooling, per posterior draw (not just the plug-in point):

individual vs pooled: €1,181 94% HDI [€1,102, €1,241] $P(>0) = 1.000$

segment vs pooled: €697 on average - lumpier across draws,

because a discrete per-segment rule only changes when a segment posterior crosses the cost line

scaled to a 100k-customer base: individual targeting is worth ~ €78,739
[€73,458, €82,716] per campaign



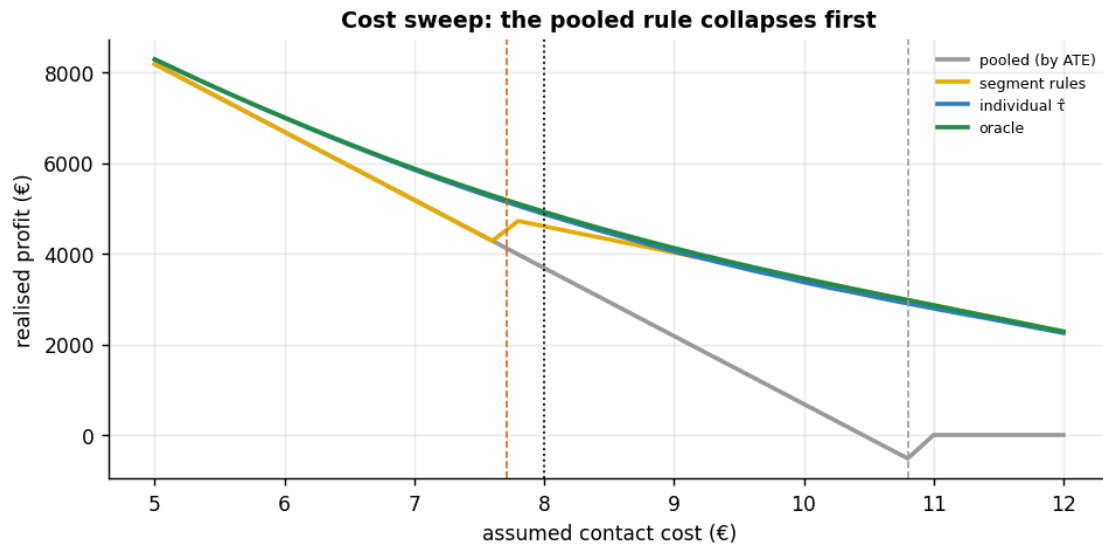
Break-even engagement (where CATE = €8), per segment:

low-value : $g^* = 0.53$ (true 0.625) 94% HDI [0.46, 0.61]

-> SEND to low-value customers only ABOVE engagement ~0.53

high-value: $g^* = -0.29$ (< 0) -> high-value clear €8 at EVERY engagement level
(always SEND)

Cost sweep €5-€12: the segment rule drops low-value once cost passes its CATE (€7.7), but the pooled rule treats EVERYONE up to the ATE (€10.8) then flips to treat-NOBODY - surrendering all the high-value profit above €10.8, exactly where segment and individual targeting keep earning.



How to read the cost sweep. The four policies earn nearly identical profit only at very low cost, where “email everyone” is optimal. As the contact cost rises they **peel apart at their decision thresholds**: the segment rule stops mailing low-value customers once cost crosses that segment’s CATE (€7.7, orange), while the **pooled rule keeps mailing everyone until cost passes the pooled ATE (€10.8, grey)** — then **collapses to mailing no-one**, forfeiting the high-value profit that individual and segment targeting still capture. That brittleness is the cost of one-size-fits-all: a single averaged decision has no way to keep the profitable segment once the *average* stops clearing cost.

The break-even engagement turns the same €8 logic into an operational cutoff: this sample’s fitted low-value line crosses €8 at engagement **~0.53** (94% HDI [0.46, 0.61]) — *mail a low-value customer only once they are more engaged than that*. The true crossing is 0.625; the fitted cutoff sits a touch lower because this sample’s low-value CATE runs ~€0.7 high (the small solid-vs-dashed gap in the moderation panel), and a break-even — a **ratio** of two estimated coefficients — amplifies that error near the crossing. That is exactly why you carry the interval, not the point, into the decision — with eyes open: in this very sample even the 94% HDI misses the true 0.625 (ratio break-evens inherit *amplified* uncertainty near a crossing), while §5b’s fresh-sample refits show the underlying CATEs are centred — and §5b’s g^* line now measures the ratio itself (bias and 94%-HDI coverage across seeds, printed there). So read the printed band as an honest *starting* range for a cutoff you would confirm on a holdout, not as a guarantee.

0.5.1 6b · The one-paragraph decision

To the CMO. The email’s effect is genuinely different across the base — the value-tier gap ($\beta_3 \approx \text{€}8$) is real with near-certainty, and responsiveness climbs with engagement — so a one-size-fits-all send provably leaves money on the table. **High-value customers: always send.** Their effect clears the €8 contact cost at every engagement level, with $P(\text{effect} > \text{cost})$ effectively 1. **Low-value customers: send only above the break-even engagement** printed in Step 6 (~ 0.5 on this sample, with a 94% band of roughly +/-0.1) — and confirm that cutoff on a holdout before hard-coding it, because a ratio

break-even is the fragile estimand of this analysis (§5b measured it; Step 6’s read-out explains it). Moving from a blanket policy to per-customer targeting is worth on the order of €1.2k on this 1,500-customer base — ~ €80k per 100k customers — and that value is sign-certain across the posterior (the printed HDI). Where we are *not* sure — the low-value, mid-engagement region — the next action is a properly sized A/B test (§5c prints the required n), not a rollout. **Act where we’re sure; measure where we’re not.**

The same decision, machine-readable — every number computed from this run, none typed by hand:

```
{
  "segment_cate_low_eur": 7.71,
  "segment_cate_low_hdi90": [
    7.1,
    8.4
  ],
  "segment_cate_high_eur": 15.79,
  "segment_cate_high_hdi90": [
    15.01,
    16.69
  ],
  "gap_b_ev_eur": 8.08,
  "P_gap_gt_0": 1.0,
  "break_even_engagement_low": 0.529,
  "break_even_hdi94": [
    0.461,
    0.612
  ],
  "profit_pooled_eur": 3682.0,
  "profit_segment_rules_eur": 4602.0,
  "profit_individual_eur": 4871.0,
  "profit_oracle_eur": 4923.0,
  "value_individual_vs_pooled_eur": 1181.0,
  "value_individual_vs_pooled_hdi94": [
    1102.0,
    1241.0
  ],
  "P_value_gt_0": 1.0,
  "value_per_100k_customers_eur": 78739.0,
  "followup_test_n_low_segment": 7758,
  "ppc_coverage_5_95": 0.901,
  "noise_segments_flagged": "1/20",
  "dag_implications_violated": "0/6"
}
```


0.6 7 · Caveats

- **Heterogeneity is only as good as the moderator you measured.** If the real driver of differential response isn't `is_high/engagement` (or a proxy), the interaction won't catch it.
- **Slice enough ways and something will “respond”.** §5c manufactured a “significant” segment out of pure noise on this very dataset. Pre-register the segments you'll act on, report posterior probabilities rather than significance stars, and shrink many-segment estimates with partial pooling (nb03).
- **The functional form is an assumption no DAG test can check.** §5d's omitted moderation sailed through the graph-falsification test while breaking every individual-level deliverable; on real data, pair the structural model with a PPC and a flexible cross-check.
- **Interaction != causing the moderator.** We're not claiming *making* someone high-value changes responsiveness — only that responsiveness *differs* across pre-existing segments.
- **Random slopes for many thin segments.** With dozens of small segments, replace the single interaction with partially-pooled random slopes so noisy segments borrow strength (see nb 03).
- **Same unconfoundedness caveat** — here email is randomized; observationally a hidden common cause of email and spend would bias every segment effect (see nb 05).